

Instacart Market Basket Analysis & Product Recommendation System

An End-to-End Business Analytics & Recommendation Engine using SQL, Python & Plotly

Table of Contents

- [1 Introduction](#)
- [2 Business Problem](#)
- [3 Dataset Overview](#)
- [4 Data Dictionary](#)
- [5 Project Workflow](#)
- [6 Import Libraries](#)
- [7 Global Configuration](#)
- [8 Load Dataset](#)
- [9 Data Quality Assessment](#)
- [10 Data Cleaning](#)
- [11 Feature Engineering](#)
- [12 Exploratory Data Analysis](#)
- [13 Recommendation Engine](#)
- [14 Business Recommendations](#)
- [15 Conclusion](#)

1 Introduction

Instacart is a North American company that operates as a grocery delivery and pick-up service. Customers order groceries from participating retailers through Instacart's website or mobile app, and personal shoppers pick and deliver the orders.

Recommendation systems are crucial for e-commerce platforms like Instacart because they:

- **Enhance customer experience:** By suggesting relevant products, recommendations make shopping easier and more enjoyable.
- **Increase sales:** Personalized suggestions lead to higher conversion rates and larger basket sizes.
- **Improve customer retention:** Satisfied customers are more likely to return.
- **Optimize inventory:** Understanding product relationships can help with stock management.

This project aims to leverage Instacart's transactional data to develop a robust recommendation engine, providing valuable insights into customer purchasing habits and product relationships.

2 Business Problem

This project is important for Instacart because it directly addresses several key business challenges:

- **Cross-selling:** How can Instacart encourage customers to buy related items they might not have considered?
- **Upselling:** How can Instacart promote higher-value or complementary products?
- **Customer retention:** How can Instacart keep customers engaged and coming back for more?
- **Average basket value:** How can Instacart increase the total value of each customer's order?
- **Repeat purchases:** How can Instacart identify products that lead to habitual buying behavior?

Business Objectives:

1. Understand customer purchasing patterns and product relationships.
2. Identify opportunities for cross-selling and upselling through market basket analysis.
3. Develop a product recommendation system to personalize the shopping experience.
4. Provide actionable business recommendations to improve sales, customer satisfaction, and retention.

6 Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
import plotly.graph_objects as go

# Suppress warnings
import warnings
warnings.filterwarnings('ignore')
```

8 Load Dataset

Loading all necessary datasets from the Kaggle dataset 'instacart-online-grocery-basket-analysis-dataset'. We will load `order_products__train.csv`, `aisles.csv`, `departments.csv`, `orders.csv`, `order_products__prior.csv`, and `products.csv`.

```
import kagglehub
from kagglehub import KaggleDatasetAdapter

orders_product_train_df = kagglehub.dataset_load(
    KaggleDatasetAdapter.PANDAS,
    "yasserh/instacart-online-grocery-basket-analysis-dataset",
    'order_products__train.csv'
)

aisles_df = kagglehub.dataset_load(
    KaggleDatasetAdapter.PANDAS,
    "yasserh/instacart-online-grocery-basket-analysis-dataset",
    'aisles.csv'
)
```

```

departments_df = kagglehub.dataset_load(
    KaggleDatasetAdapter.PANDAS,
    "yasserh/instacart-online-grocery-basket-analysis-dataset",
    'departments.csv'
)

orders_df = kagglehub.dataset_load(
    KaggleDatasetAdapter.PANDAS,
    "yasserh/instacart-online-grocery-basket-analysis-dataset",
    'orders.csv'
)

order_products__prior_df = kagglehub.dataset_load(
    KaggleDatasetAdapter.PANDAS,
    "yasserh/instacart-online-grocery-basket-analysis-dataset",
    'order_products__prior.csv'
)

products_df = kagglehub.dataset_load(
    KaggleDatasetAdapter.PANDAS,
    "yasserh/instacart-online-grocery-basket-analysis-dataset",
    'products.csv'
)

print("All datasets loaded successfully.")

```

Using Colab cache for faster access to the 'instacart-online-grocery-basket-analysis-dataset' dat
 Using Colab cache for faster access to the 'instacart-online-grocery-basket-analysis-dataset' dat
 Using Colab cache for faster access to the 'instacart-online-grocery-basket-analysis-dataset' dat
 Using Colab cache for faster access to the 'instacart-online-grocery-basket-analysis-dataset' dat
 Using Colab cache for faster access to the 'instacart-online-grocery-basket-analysis-dataset' dat
 All datasets loaded successfully.

```

print("\n--- Dataset Shapes ---")
print(f"orders_product_train_df: {orders_product_train_df.shape}")
print(f"aisles_df: {aisles_df.shape}")
print(f"departments_df: {departments_df.shape}")
print(f"orders_df: {orders_df.shape}")
print(f"order_products__prior_df: {order_products__prior_df.shape}")
print(f"products_df: {products_df.shape}")

print("\n--- Dataset Memory Usage ---")
print(f"orders_product_train_df: {orders_product_train_df.memory_usage(deep=True).sum() / (1024*1024):.2f} MB")
print(f"aisles_df: {aisles_df.memory_usage(deep=True).sum() / (1024**2):.2f} MB")
print(f"departments_df: {departments_df.memory_usage(deep=True).sum() / (1024**2):.2f} MB")
print(f"orders_df: {orders_df.memory_usage(deep=True).sum() / (1024**2):.2f} MB")
print(f"order_products__prior_df: {order_products__prior_df.memory_usage(deep=True).sum() / (1024**2):.2f} MB")
print(f"products_df: {products_df.memory_usage(deep=True).sum() / (1024**2):.2f} MB")

```

```

--- Dataset Shapes ---
orders_product_train_df: (1384617, 4)
aisles_df: (134, 2)
departments_df: (21, 2)
orders_df: (3421083, 7)
order_products__prior_df: (32434489, 4)

```

```
products_df: (49688, 4)

--- Dataset Memory Usage ---
orders_product_train_df: 42.26 MB
aisles_df: 0.01 MB
departments_df: 0.00 MB
orders_df: 332.71 MB
order_products__prior_df: 989.82 MB
products_df: 4.93 MB
```

✓ Preview of Datasets

Here are the first 5 rows of each dataset to get a quick overview of their content:

```
print("\n--- orders_product_train_df Head ---")
display(orders_product_train_df.head())

print("\n--- aisles_df Head ---")
display(aisles_df.head())

print("\n--- departments_df Head ---")
display(departments_df.head())

print("\n--- orders_df Head ---")
display(orders_df.head())

print("\n--- order_products__prior_df Head ---")
display(order_products__prior_df.head())

print("\n--- products_df Head ---")
display(products_df.head())
```



```
--- orders_product_train_df Head ---
```

	order_id	product_id	add_to_cart_order	reordered
0	1	49302	1	1
1	1	11109	2	1
2	1	10246	3	0
3	1	49683	4	0

T **B** *I* <> ⇌ Close

7 Global Configuration

7 Global Configuration

u **i** prepared soups saiaus

This section configures global settings for the notebook, including Plotly themes, font styles, colors, figure dimensions, Pandas display options, and warning suppression. These configurations ensure a consistent and aesthetically pleasing presentation of data visualizations and table outputs.

```
import plotly.io as pio

# --- Plotly Theme & Layout ---
plotly_colors = {
    'primary': '#0AAD05',
    'secondary': '#FF7009',
    'dark_green': '#003D29',
    'background': '#FFFFFF',
    'cards': '#F7F7F7',
    'text': '#2F2F2F'
}

pio.templates['instacart_template'] = go.layout.Template(
    layout=go.Layout(
        font_family='Inter, Poppins, Source Sans Pro, sans-serif',
        font_color=plotly_colors['text'],
        title_font_color=plotly_colors['dark_green'],
        plot_bgcolor=plotly_colors['background'],
        paper_bgcolor=plotly_colors['background'],
        colorway=[plotly_colors['primary'], plotly_colors['secondary'], plotly_colors['dark_gree
        hovermode='closest',
        hoverlabel=dict(
            bgcolor='white',
            font_size=12,
            font_family='Inter, Poppins, Source Sans Pro, sans-serif'
        ),
        xaxis=dict(
            showgrid=False,
            linecolor=plotly_colors['cards'],
            linewidth=1,
            ticks='outside',
            tickfont=dict(size=12, color=plotly_colors['text'])
        ),
        yaxis=dict(
            showgrid=True,
            gridcolor=plotly_colors['cards'],
            linecolor=plotly_colors['cards'],
```

```

        linewidth=1,
        ticks='outside',
        tickfont=dict(size=12, color=plotly_colors['text'])
    ),
    # Customize legend
    legend=dict(
        orientation='h',
        yanchor='bottom',
        y=1.02,
        xanchor='right',
        x=1,
        bgcolor='rgba(255,255,255,0.7)',
        bordercolor=plotly_colors['cards'],
        borderwidth=1,
        font=dict(size=10, color=plotly_colors['text'])
    )
)
pio.templates.default = 'plotly_white+instacart_template'

# --- Pandas Options ---
pd.set_option('display.max_columns', None)          # Display all columns
pd.set_option('display.max_rows', 100)             # Display up to 100 rows
pd.set_option('display.width', 1000)               # Set display width for better readability
pd.set_option('display.float_format', '{:.2f}'.format) # Format float values to 2 decimal places

# --- Random Seed ---
np.random.seed(42)

print("Global configurations applied: Plotly theme, Pandas display options, and random seed set.")

```

✓ 9 Data Quality Assessment

This section performs a comprehensive data quality assessment to identify and understand potential issues such as missing values, duplicate entries, incorrect datatypes, and the uniqueness of key identifiers across all loaded datasets. Addressing these issues is crucial for ensuring the reliability and accuracy of subsequent analyses and model building.

T B I <> ↺ ↻ 🖼️ 🔍 ⌵ ⌶ — Ψ 😊 ☰ [Close](#)

Missing Values

We will check for the presence of missing values in each dataset to understand their completeness. Missing data can impact analysis and model performance, so it's important to identify them early.

Missing Values

We will check for the presence of missing values in each dataset to understand their completeness. Missing data can impact analysis and model performance, so it's important to identify them early.

◆ Gemini

```

def check_missing_values(df, df_name):
    missing_counts = df.isnull().sum()
    missing_percentages = (df.isnull().sum() / len(df)) * 100

```

```

missing_info = pd.DataFrame({
    'Missing Count': missing_counts,
    'Missing Percentage (%)': missing_percentages
})
missing_info = missing_info[missing_info['Missing Count'] > 0].sort_values(by='Missing Count')

print(f"\n--- Missing Values in {df_name} ---")
if missing_info.empty:
    print("No missing values.")
else:
    display(missing_info)

check_missing_values(orders_product_train_df, 'orders_product_train_df')
check_missing_values(aisles_df, 'aisles_df')
check_missing_values(departments_df, 'departments_df')
check_missing_values(orders_df, 'orders_df')
check_missing_values(order_products__prior_df, 'order_products__prior_df')
check_missing_values(products_df, 'products_df')

```

```

# Install dependencies as needed:
# pip install kagglehub[pandas-datasets]
import kagglehub
from kagglehub import KaggleDatasetAdapter

# Set the path to the file you'd like to load
file_path = "order.csv"

# Load the latest version
df = kagglehub.load_dataset(
    KaggleDatasetAdapter.PANDAS,
    "yasserh/instacart-online-grocery-basket-analysis-dataset",
    file_path,

print("First 5 records:", df.head())

```

File `"/tmp/ipykernel_981/2120242314.py"`, line 11
`KaggleDatasetAdapter.PANDAS,`
`^`
IndentationError: unexpected indent

Next steps: [Explain error](#)

```

import kagglehub
from kagglehub import KaggleDatasetAdapter

# Set the path to the file you'd like to load
file_path = "order_products__train.csv"

# Load the latest version
df = kagglehub.dataset_load(
    KaggleDatasetAdapter.PANDAS,
    "yasserh/instacart-online-grocery-basket-analysis-dataset",
)

print("First 5 records:", df.head())

```


Using Colab cache for faster access to the 'instacart-online-grocery-basket-analysis-dataset' dataset
 First 5 records:

	order_id	product_id	add_to_cart_order	reordered
0	1	49302	1	1
1	1	11109	2	1
2	1	10246	3	0
3	1	49683	4	0
4	1	43633	5	1

```
import kagglehub
from kagglehub import KaggleDatasetAdapter

# Set the path to the file you'd like to load
file_path = "order_products__train.csv"

# Load the latest version
df = kagglehub.dataset_load(
    KaggleDatasetAdapter.PANDAS,
    "yasserh/instacart-online-grocery-basket-analysis-dataset",
    file_path
)

print("First 5 records:", df.head())
```

Using Colab cache for faster access to the 'instacart-online-grocery-basket-analysis-dataset' dataset
 First 5 records:

	order_id	product_id	add_to_cart_order	reordered
0	1	49302	1	1
1	1	11109	2	1
2	1	10246	3	0
3	1	49683	4	0
4	1	43633	5	1

```
import kagglehub
from kagglehub import KaggleDatasetAdapter

# Set the path to the file you'd like to load
file_path = "order_products__train.csv"

# Load the latest version
df = kagglehub.dataset_load(
    KaggleDatasetAdapter.PANDAS,
    "yasserh/instacart-online-grocery-basket-analysis-dataset",
    file_path
)

print("First 5 records:", df.head())
```

Using Colab cache for faster access to the 'instacart-online-grocery-basket-analysis-dataset' dataset
 First 5 records:

	order_id	product_id	add_to_cart_order	reordered
0	1	49302	1	1
1	1	11109	2	1
2	1	10246	3	0
3	1	49683	4	0
4	1	43633	5	1

```
import kagglehub
from kagglehub import KaggleDatasetAdapter

orders_product_train_df = kagglehub.dataset_load(
```

```

KaggleDatasetAdapter.PANDAS,
"yasserh/instacart-online-grocery-basket-analysis-dataset",
'order_products__train.csv'
)

aisles_df = kagglehub.dataset_load(
    KaggleDatasetAdapter.PANDAS,
    "yasserh/instacart-online-grocery-basket-analysis-dataset",
    'aisles.csv'
)

departments_df = kagglehub.dataset_load(
    KaggleDatasetAdapter.PANDAS,
    "yasserh/instacart-online-grocery-basket-analysis-dataset",
    'departments.csv'
)

orders_df = kagglehub.dataset_load(
    KaggleDatasetAdapter.PANDAS,
    "yasserh/instacart-online-grocery-basket-analysis-dataset",
    'orders.csv'
)

order_products__prior_df = kagglehub.dataset_load(
    KaggleDatasetAdapter.PANDAS,
    "yasserh/instacart-online-grocery-basket-analysis-dataset",
    'order_products__prior.csv'
)

products_df = kagglehub.dataset_load(
    KaggleDatasetAdapter.PANDAS,
    "yasserh/instacart-online-grocery-basket-analysis-dataset",
    'products.csv'
)

print("First 5 records:", df.head())

```

```

Downloading to /root/.cache/kagglehub/datasets/yasserh/instacart-online-grocery-basket-analysis-d
100%|██████████| 2.54k/2.54k [00:00<00:00, 1.51MB/s]
Using Colab cache for faster access to the 'instacart-online-grocery-basket-analysis-dataset' dat
Using Colab cache for faster access to the 'instacart-online-grocery-basket-analysis-dataset' dat
Using Colab cache for faster access to the 'instacart-online-grocery-basket-analysis-dataset' dat
Using Colab cache for faster access to the 'instacart-online-grocery-basket-analysis-dataset' dat
First 5 records:
  order_id  product_id  add_to_cart_order  reordered
0         1      49302                 1          1
1         1      11109                 2          1
2         1      10246                 3          0
3         1      49683                 4          0
4         1      43633                 5          1

```

```
products_df.head()
```

